

Une ligne, quatre stations, **aucun arrêt sauté**

◆ Authentification ◆ API propre ◆ IA & asynchrone ◆ Tests



STATION 01 – QUI ES-TU ?

Authentification — Sanctum

Une API n'a pas de mémoire : chaque requête doit prouver son identité.

- ◆ Le **Bearer token** est comme un badge d'accès. Il a la forme `id|chaîne` et il n'est montré qu'une seule fois, au moment de sa création.
- ◆ Le serveur ne garde jamais le token en clair dans la base de données : il stocke uniquement son **hash SHA-256** dans la table `personal_access_tokens`.
- ◆ Au login, la méthode `createToken` fabrique le badge. Elle existe grâce au trait `HasApiTokens` ajouté au model User.
- ◆ À chaque requête, le middleware `auth:sanctum` vérifie le badge. Si le badge est absent ou invalide, la réponse est **401**.

PIÈGE Si tu ajoutes un `Hash::make` dans le register, le mot de passe est haché deux fois et le login échoue. Laravel 13 le hache déjà tout seul grâce au cast `hashed`.



API REST propre

Une API qui répond ne suffit pas : elle doit renvoyer les bonnes données, dans le bon format, avec le bon status code.

- ◆ La **Form Request** valide les données avant que le controller s'exécute. Si les données sont invalides, elle renvoie un **422** propre au lieu de laisser passer une erreur SQL.
- ◆ L'**API Resource** choisit exactement quels champs sortent dans le JSON. Écrire `return Model::all()` expose tout le model, c'est une fuite de données.
- ◆ Chaque status code raconte ce qui s'est passé : **200** pour une lecture réussie, **201** pour une création, **401** quand le badge manque, **422** quand les données sont invalides.



STATION 03 – ÇA MARCHE, MAIS C'EST LENT

IA fiable & génération asynchrone

Une seule histoire en trois temps : l'appel IA marche, mais il fige la requête, alors on le déplace dans la queue.

- ◆ Le **structured output** impose un contrat à l'IA : grâce au `schema()`, la réponse revient toujours avec les mêmes champs et les bons types.
- ◆ Les **Casts** gardent les données typées dans la base de données : `array` pour les tableaux, un **backed enum** pour le statut.
- ◆ Un appel lent ne reste jamais dans la requête : on le met dans un **Job**. Le controller fait `dispatch`, répond **202** tout de suite, et le **worker** traite en arrière-plan.
- ◆ C'est comme au **ferran** : tu déposes ta pâte, tu repars faire tes courses, et tu reviens chercher ton pain une fois cuit.

PIÈGE N°1 Si le worker n'est pas lancé avec `queue:work`, rien ne se passe : le Job reste en attente dans la table `jobs`.



STATION 04 – LE CODE QUI VÉRIFIE TON CODE

Tests — Pest

Un test s'écrit une fois et se rejoue autant de fois qu'on veut, avec une seule commande : `php artisan test`.

- ◆ On teste le **comportement** de l'API : l'endpoint renvoie un 200 avec la bonne structure JSON, un 401 sans badge, un 422 avec des données invalides.
- ◆ `Sanctum::actingAs` connecte un utilisateur de test, et les **factories** fabriquent les données dont le test a besoin.
- ◆ Avec `Queue::fake` et `assertPushed`, on vérifie que le Job est bien dispatché, **sans appeler la vraie IA**.

PROLONGEMENT EN TRAVAUX – SEMAINE 2

De ton code à la production

Git avancé en équipe

CI — tests à chaque push

Déploiement Azure

Sans vos tests de cette semaine, la CI de la semaine prochaine n'a rien à lancer.